C++ Low-Level Internals and Common Pitfalls

Theory, Edge Cases & Code Examples

- ✓ Object Memory Layout with Vtables
- ✓ Alignment and Padding Optimization
- ✓ Lifetime Management and Dangling References
- ✓ Const Correctness
- ✓ Undefined Behavior (UB)

→ Swipe to learn →

C++ Low-Level Internals and Common Pitfalls

Theory

📌 1. Object Memory Layout with Vtables

Memory Layout Components:

Component	Location	Purpose	Size (typical)
vptr	Beginning (if virtual functions exist)	Points to vtable for dynamic dispatch	8 bytes (64-bit)
Base class members	After vptr	Inherited data from base classes	Variable
Derived class members	After base members	Class-specific data	Variable
Padding bytes	Between members	Satisfy alignment requirements	0-7 bytes typically

How Virtual Dispatch Works:

1. Each class with virtual functions has a **static vtable** (virtual function table)
2. Each object has a **vptr** pointing to its class's vtable
3. Virtual function call: `obj->foo()` → `obj->vptr[index_of_foo]()`
4. Runtime overhead: One pointer dereference per virtual call

Object Slicing (Common Pitfall):

```
Derived d;
Base b = d;  // ❌ Slicing: loses Derived parts, resets vptr to Base vtable
b.foo();     // Calls Base::foo, not Derived::foo

Base* bp = &d;  // ✅ Correct: maintains polymorphism
bp->foo();      // Calls Derived::foo via virtual dispatch
```

C++ Low-Level Internals and Common Pitfalls

Theory

📌 2. Alignment and Padding Optimization (1/2)

Alignment Requirements:

Type	Typical Alignment	Must be multiple of
`char`	1 byte	1
`short`	2 bytes	2
`int`	4 bytes	4
`long` / `pointer` (64-bit)	8 bytes	8
`double`	8 bytes	8

Padding Example:

```
// ❌ Poor layout: 12 bytes with 6 wasted bytes
struct Bad {
    char a;    // 1 byte + 3 padding
    int b;     // 4 bytes
    char c;    // 1 byte + 3 padding
};  // Total: 12 bytes

// ✅ Optimized layout: 8 bytes with 2 wasted bytes
struct Good {
    int b;     // 4 bytes
    char a;    // 1 byte
    char c;    // 1 byte
    // 2 bytes padding at end
};  // Total: 8 bytes (33% smaller!)
```

C++ Low-Level Internals and Common Pitfalls

Theory

📌 2. Alignment and Padding Optimization (2/2)

Cache Line Alignment for Multithreading:

Use Case	Alignment	Purpose
False sharing prevention	`alignas(64)`	Separate variables to different cache lines
SIMD operations	`alignas(16)` or `alignas(32)`	Required for vectorized instructions
General structs	Natural (automatic)	Balance size and performance

---

C++ Low-Level Internals and Common Pitfalls

Theory

📌 3. Lifetime Management and Dangling References

Dangling Reference Scenarios:

Scenario	Risk	Example
Returning reference to local variable	❌ UB	`const std::string& foo() { std::string s; return s; }`
Temporary not extended	❌ UB	`const std::string& r = get_temp();` (temp destroyed)
Use-after-free	❌ UB	`delete p; *p = 5;`
Lifetime extension (safe)	✅ Safe	`const std::string& r = std::string("temp");`

Lifetime Extension Rules:

- ✅ Works: Binding unnamed temporary to `const` lvalue reference in same scope
- ❌ Doesn't work: Returning reference from function (temp destroyed before caller)
- ❌ Doesn't work: Named local variables (not temporaries)

C++ Low-Level Internals and Common Pitfalls

Theory

📌 4. Const Correctness

Const Pointer Variations:

Declaration	Modify `*p`?	Reassign `p`?	Read as
`const int* p`	❌ No	✅ Yes	Pointer to const int
`int* const p`	✅ Yes	❌ No	Const pointer to int
`const int* const p`	❌ No	❌ No	Const pointer to const int

Mutable Keyword (Logical vs Bitwise Constness):

Constness Type	Meaning	Use Case
Bitwise	No bit of object changes	Default C++ behavior
Logical	Observable state unchanged	Internal caching, lazy evaluation, mutex locks

```
struct Cache {
    mutable int cached_value;  // Can modify in const functions
    int compute() const {
        cached_value = expensive_calc();  // ✅ Allowed
        return cached_value;
    }
};
```

C++ Low-Level Internals and Common Pitfalls

Theory

📌 5. Undefined Behavior (UB)

Common UB Sources:

Category	Example	Typical Result
Null pointer	`int* p = nullptr; *p = 5;`	Segfault
Out-of-bounds	`int arr[5]; arr[10] = 0;`	Memory corruption
Use-after-free	`delete p; *p = 5;`	Heap corruption
Signed overflow	`INT_MAX + 1`	UB (unpredictable)
Uninitialized variable	`int x; cout << x;`	Garbage value
Data race	Two threads modify same variable	Corruption

UB Detection Tools:

- **AddressSanitizer (ASan)**: Detects memory errors
- **UndefinedBehaviorSanitizer (UBSan)**: Detects UB at runtime
- **Static analyzers**: Clang-Tidy, Coverity
- **Compiler warnings**: `-Wall -Wextra -Werror`

C++ Low-Level Internals and Common Pitfalls

Theory

📌 6. Copy Semantics: Shallow vs Deep Copy

Copy Type	Pointer Behavior	Result	Risk
Shallow	Copies pointer value	Both objects share same memory	Double delete, dangling pointers
Deep	Allocates new memory, copies data	Each object owns independent memory	Safe, requires Rule of Three/Five

Rule of Three / Five / Zero:

Rule	Components	When to Use
Rule of Three (C++98)	Destructor, Copy constructor, Copy assignment	Managing raw resources
Rule of Five (C++11+)	+ Move constructor, Move assignment	Add move semantics for efficiency
Rule of Zero (modern)	None (use smart pointers)	**Preferred**: Automatic management

```
// ✅ Modern approach: Rule of Zero
class GoodString {
    std::unique_ptr<char[]> data;  // Smart pointer handles everything
    // No need for custom destructor, copy/move operations
};
```

C++ Low-Level Internals and Common Pitfalls

Theory

📌 7. Virtual Inheritance and the Diamond Problem

Diamond Problem:

```
    VBase
    /   \
  A       B
    \   /
      C
```

Inheritance Type	VBase Copies	Ambiguity	Object Size	Performance
Normal	2 copies (via A and B)	❌ Ambiguous access	Smaller	Faster
Virtual	1 shared copy	✅ Unambiguous	Larger (+8 bytes/pointer)	Slightly slower

Virtual Inheritance Implementation:

- Adds **virtual base pointer** in derived classes
- Most derived class constructs virtual base (not intermediate classes)
- Requires indirection to access virtual base members
- Constructor order: Virtual bases → Direct bases → Derived class

Typical Size Impact:

```
struct Normal : Base { int y; };     // ~12 bytes (Base + y)
struct Virtual : virtual Base { int y; };  // ~20 bytes (vptr + Base + y)
```

C++ Low-Level Internals and Common Pitfalls

Edge Cases

📌 1. Virtual Function Calls in Constructors/Destructors

```
struct Base {
    Base() { foo(); }  // Calls Base::foo, NOT Derived::foo!
    virtual void foo() { std::cout << "Base::foo\n"; }
};

struct Derived : Base {
    void foo() override { std::cout << "Derived::foo\n"; }
};

int main() {
    Derived d;  // Prints "Base::foo"
}
```

During construction, the vptr points to the Base vtable until Base's constructor completes. Similarly, during destruction, the vptr is reset to Base before Derived's destructor runs. This prevents calling pure virtual functions or accessing derived members that haven't been constructed yet (or have already been destroyed).

Key takeaway: Virtual dispatch is disabled during construction/destruction to prevent accessing uninitialized or destroyed derived class members.

C++ Low-Level Internals and Common Pitfalls

Edge Cases

📌 2. Padding Can Change with Member Order

```
struct Bad {
    char a;    // 1 byte
    int b;     // 4 bytes (needs 4-byte alignment)
    char c;    // 1 byte
};  // sizeof(Bad) = 12 due to padding

struct Good {
    int b;     // 4 bytes
    char a;    // 1 byte
    char c;    // 1 byte
};  // sizeof(Good) = 8 (a and c share padding after b)
```

Memory layout of `Bad`:

```
[a][pad][pad][pad][b][b][b][b][c][pad][pad][pad]
 1   3 bytes pad    4 bytes     1   3 bytes pad
```

Memory layout of `Good`:

```
[b][b][b][b][a][c][pad][pad]
 4 bytes     1  1   2 bytes pad
```

The `Good` struct is 33% smaller simply by reordering members. In an embedded system allocating millions of sensor readings, this optimization could save significant memory.

Key takeaway: Always order struct members from largest to smallest type to minimize padding waste.

C++ Low-Level Internals and Common Pitfalls

Edge Cases

📌 3. Const Reference Lifetime Extension vs Local Variables

```
// ✅ Lifetime extended - SAFE
const std::string& safe() {
    const std::string& r = std::string("temp");  // Temporary lifetime extended
    return r;  // STILL UNSAFE! r goes out of scope, temporary is destroyed
}

// ❌ Dangling reference - UNSAFE
const std::string& unsafe() {
    std::string s = "hello";  // Named local variable
    return s;  // s is destroyed, returning dangling reference
}

// ✅ Safe usage
void correct() {
    const std::string& r = std::string("temp");  // Lifetime extended to r's scope
    std::cout << r;  // Safe to use here
}  // Temporary destroyed when r goes out of scope
```

The lifetime extension rule only applies to **binding a temporary to a const reference in the same scope**. Returning the reference from a function doesn't extend the lifetime beyond the function's scope.

Key takeaway: Lifetime extension works for temporaries bound to const references, but returning references from functions still creates dangling references.

C++ Low-Level Internals and Common Pitfalls

Edge Cases

📌 4. Mutable and Bitwise vs Logical Constness

```
struct Cache {
    mutable int cached_value;
    mutable bool is_cached;

    int expensive_compute() const {  // Const function!
        if (!is_cached) {
            cached_value = /* expensive computation */;
            is_cached = true;  // Modifying mutable member in const function
        }
        return cached_value;
    }
};
```

Bitwise constness means no bits of the object are changed. **Logical constness** means the observable state doesn't change, even if internal cache/optimization members are modified. The `mutable` keyword allows logical constness while violating bitwise constness.

Common use cases:
- Lazy evaluation and caching
- Mutex locks in const member functions (thread safety)
- Reference counting in smart pointers

Key takeaway: Use mutable for members that don't affect the logical state but need to be modified in const contexts (caching, locking, counters).

C++ Low-Level Internals and Common Pitfalls

Edge Cases

📌 5. Virtual Inheritance Constructor Order

```
struct VBase {
    VBase() { std::cout << "VBase\n"; }
};

struct A : virtual VBase {
    A() { std::cout << "A\n"; }
};

struct B : virtual VBase {
    B() { std::cout << "B\n"; }
};

struct C : A, B {
    C() { std::cout << "C\n"; }
};

int main() {
    C obj;
    // Output: VBase, A, B, C
}
```

With virtual inheritance, the **most derived class** (C) constructs the virtual base (VBase) directly, not A or B. Construction order:

1. Virtual base classes (VBase)
2. Direct base classes in declaration order (A, then B)
3. Derived class (C)

This is different from normal inheritance where each base class constructs its own bases. With virtual inheritance, intermediate bases (A and B) **skip** constructing VBase because C handles it.

Key takeaway: Virtual base classes are constructed by the most derived class, ensuring only one shared instance exists.

C++ Low-Level Internals and Common Pitfalls

Code Examples

📌 1. Object Memory Layout with vtable

The memory layout of `Derived` is approximately:

When assigning to `Base b2`, only the vptr and x are copied, losing y and the connection to Derived's vtable. This demonstrates why passing polymorphic objects by pointer or reference is crucial.

```
#include <iostream>

struct Base {
    int x = 1;
    virtual void foo() { std::cout << "Base::foo\n"; }
    virtual ~Base() = default;
};

struct Derived : Base {
    int y = 2;
    void foo() override { std::cout << "Derived::foo\n"; }
};

int main() {
    std::cout << "sizeof(Base): " << sizeof(Base) << "\n";      // 16 bytes (8-byte vptr + 4-byte int + padding)
    std::cout << "sizeof(Derived): " << sizeof(Derived) << "\n"; // 16 bytes (vptr + x + y + padding)

    Derived d;
    Base* b = &d;
    b->foo();  // ✅ Prints "Derived::foo" via virtual dispatch

    Base b2 = d;  // ❌ Object slicing!
    b2.foo();     // Prints "Base::foo" - lost Derived vtable
}
```

```
['[vptr (8 bytes)][x (4 bytes)][y (4 bytes)]']
```

C++ Low-Level Internals and Common Pitfalls

Code Examples

📌 2. Alignment and Padding Optimization

In autonomous driving systems, sensor data structures are often allocated millions of times. Optimizing from 12 to 8 bytes per structure saves 33% memory, potentially allowing more data to fit in CPU cache for faster processing. Cache line alignment prevents false sharing when different threads access different variables on the same cache line.

```
#include <iostream>

// ❌ Poorly ordered - 12 bytes
struct BadLayout {
    char a;    // 1 byte
    int b;     // 4 bytes (needs 4-byte alignment)
    char c;    // 1 byte
};

// ✅ Well ordered - 8 bytes
struct GoodLayout {
    int b;     // 4 bytes
    char a;    // 1 byte
    char c;    // 1 byte
    // 2 bytes padding at end
};

// Cache line alignment for multithreading
struct alignas(64) CacheLineAligned {
    int data;
    // Padded to 64 bytes to prevent false sharing
};

int main() {
    std::cout << "BadLayout: " << sizeof(BadLayout) << "\n";    // 12
    std::cout << "GoodLayout: " << sizeof(GoodLayout) << "\n";  // 8
    std::cout << "alignof(int): " << alignof(int) << "\n";      // 4
    std::cout << "CacheLineAligned: " << sizeof(CacheLineAligned) << "\n";  // 64
}
```

C++ Low-Level Internals and Common Pitfalls

Code Examples

📌 3. Dangling References and Lifetime Extension

A common mistake in autonomous driving code is storing references to temporary sensor readings. Always ensure sensor data is owned (by value or smart pointer) rather than referenced, unless you can prove the source data outlives the reference.

```
#include <iostream>
#include <string>

// ❌ UNSAFE - dangling reference to local variable
const std::string& danger1() {
    std::string s = "local";
    return s;  // s destroyed, dangling reference
}

// ❌ UNSAFE - returning reference to temporary doesn't save it
const std::string& danger2() {
    return std::string("temp");  // Temporary destroyed immediately
}

// ✅ SAFE - lifetime extended within scope
void safe_usage() {
    const std::string& r = std::string("temp");  // Lifetime extended to r's scope
    std::cout << r << "\n";  // Safe to use
}  // Temporary destroyed here

// ✅ SAFE - returning by value
std::string safe_return() {
    std::string s = "local";
    return s;  // Copy or move (usually move-optimized)
}

int main() {
    // const std::string& bad = danger1();  // UB - accessing freed memory
    // std::cout << bad;  // Crash or garbage

    safe_usage();  // Works correctly

    std::string good = safe_return();  // Safe
    std::cout << good << "\n";
}
```

C++ Low-Level Internals and Common Pitfalls

Code Examples

📌 4. Const Correctness and Mutable

This pattern is common in real-time systems where expensive computations should be cached but the object is logically const from the caller's perspective. Mutable allows optimization without breaking const correctness.

```
#include <iostream>
#include <cmath>

struct SensorData {
    double raw_value;
    mutable double cached_processed;
    mutable bool is_cached;

    SensorData(double v) : raw_value(v), cached_processed(0), is_cached(false) {}

    // ✅ Const function with internal caching
    double get_processed() const {
        if (!is_cached) {
            cached_processed = std::sqrt(raw_value * raw_value + 1.0);  // Expensive
            is_cached = true;
        }
        return cached_processed;
    }

    // ❌ This would be an error without mutable
    // void compute() const { cached_processed = 5.0; }  // Error: modifying non-mutable member
};

void process_sensor(const SensorData& sensor) {
    // sensor is const, but get_processed() can still update cache
    double value = sensor.get_processed();
    std::cout << "Processed: " << value << "\n";
}

int main() {
    const SensorData sensor(42.0);
    process_sensor(sensor);  // First call computes and caches
    process_sensor(sensor);  // Second call uses cached value
}
```

C++ Low-Level Internals and Common Pitfalls

Code Examples

📌 5. Shallow vs Deep Copy and Rule of Three (1/2)

In modern C++, prefer `std::string` or smart pointers to avoid manual memory management. If you must use raw pointers, implement Rule of Five (add move constructor and move assignment) for efficiency.

```
#include <iostream>
#include <cstring>

// ❌ Broken - shallow copy causes double delete
class BadString {
    char* data;
public:
    BadString(const char* s) : data(new char[strlen(s) + 1]) {
        strcpy(data, s);
    }
    ~BadString() { delete[] data; }
    // Missing: copy constructor and assignment operator
    // Default shallow copy leads to double delete!
};

// ✅ Correct - Rule of Three implemented
class GoodString {
    char* data;
public:
    GoodString(const char* s) : data(new char[strlen(s) + 1]) {
        strcpy(data, s);
    }

    // Copy constructor - deep copy
    GoodString(const GoodString& other) : data(new char[strlen(other.data) + 1]) {
        strcpy(data, other.data);
    }

    // Copy assignment operator - deep copy
    GoodString& operator=(const GoodString& other) {
        if (this != &other) {
            delete[] data;  // Clean up old data
            data = new char[strlen(other.data) + 1];
            strcpy(data, other.data);
        }
        return *this;
// ...continues on next slide
```

C++ Low-Level Internals and Common Pitfalls

Code Examples

📌 5. Shallow vs Deep Copy and Rule of Three (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }

    ~GoodString() { delete[] data; }
};

int main() {
    // BadString b1("test");
    // BadString b2 = b1;  // Shallow copy - both point to same memory
    // // When b2 destructs, deletes memory
    // // When b1 destructs, deletes already-freed memory -> crash!

    GoodString g1("test");
    GoodString g2 = g1;  // Deep copy - independent memory
    // Both destruct safely with their own memory
}
```

C++ Low-Level Internals and Common Pitfalls

Code Examples

📌 6. Virtual Inheritance and Diamond Problem (1/2)

Note that `GoodHybrid`'s constructor must explicitly initialize `VehicleBase` even though `GoodCar` and `GoodTruck` also inherit from it. The most derived class is responsible for constructing virtual base classes. This is essential in complex automotive software architectures where multiple systems (braking, steering, powertrain) share common base interfaces.

```
#include <iostream>

// The diamond problem
struct VehicleBase {
    int id;
    VehicleBase(int i) : id(i) { std::cout << "VehicleBase(" << id << ")\n"; }
};

// ❌ Without virtual inheritance - two copies of VehicleBase
struct BadCar : VehicleBase {
    BadCar(int i) : VehicleBase(i) { std::cout << "BadCar\n"; }
};

struct BadTruck : VehicleBase {
    BadTruck(int i) : VehicleBase(i) { std::cout << "BadTruck\n"; }
};

struct BadHybrid : BadCar, BadTruck {
    BadHybrid(int i) : BadCar(i), BadTruck(i) { std::cout << "BadHybrid\n"; }
    // ERROR: ambiguous access
    // void print() { std::cout << id; }  // Which id? BadCar::id or BadTruck::id?
};

// ✅ With virtual inheritance - one shared VehicleBase
struct GoodCar : virtual VehicleBase {
    GoodCar(int i) : VehicleBase(i) { std::cout << "GoodCar\n"; }
};

struct GoodTruck : virtual VehicleBase {
    GoodTruck(int i) : VehicleBase(i) { std::cout << "GoodTruck\n"; }
};

struct GoodHybrid : GoodCar, GoodTruck {
    GoodHybrid(int i) : VehicleBase(i), GoodCar(i), GoodTruck(i) {
        std::cout << "GoodHybrid\n";
// ...continues on next slide
```

C++ Low-Level Internals and Common Pitfalls

Code Examples

📌 6. Virtual Inheritance and Diamond Problem (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }
    // ✅ Unambiguous access
    void print() { std::cout << "Vehicle ID: " << id << "\n"; }
};

int main() {
    std::cout << "Creating GoodHybrid:\n";
    GoodHybrid h(42);
    // Output: VehicleBase(42), GoodCar, GoodTruck, GoodHybrid
    h.print();  // Prints "Vehicle ID: 42"

    std::cout << "Size without virtual inheritance: " << sizeof(BadHybrid) << "\n";
    std::cout << "Size with virtual inheritance: " << sizeof(GoodHybrid) << "\n";
    // Virtual inheritance adds pointer overhead
}
```

C++ Low-Level Internals and Common Pitfalls

Code Examples

📌 7. Undefined Behavior Examples (1/2)

In safety-critical automotive systems, undefined behavior is unacceptable. Use static analyzers (Clang-Tidy, Coverity), runtime sanitizers (ASan, UBSan), and defensive programming (bounds checking, assertions) to catch UB before it reaches production.

```
#include <iostream>

void demonstrate_ub() {
    // ❌ UB: Null pointer dereference
    int* p = nullptr;
    // *p = 5;  // Crash or UB

    // ❌ UB: Out of bounds array access
    int arr[5] = {1, 2, 3, 4, 5};
    // std::cout << arr[10];  // UB - may crash, may read garbage

    // ❌ UB: Use after free
    int* heap = new int(42);
    delete heap;
    // std::cout << *heap;  // UB - accessing freed memory

    // ❌ UB: Signed integer overflow
    int max = 2147483647;  // INT_MAX
    // int overflow = max + 1;  // UB - signed overflow

    // ✅ Defined: Unsigned integer overflow (wraps around)
    unsigned int umax = 4294967295u;  // UINT_MAX
    unsigned int wrap = umax + 1;  // Defined: wraps to 0
    std::cout << "Unsigned wrap: " << wrap << "\n";

    // ❌ UB: Uninitialized variable
    int uninit;
    // std::cout << uninit;  // UB - may be any value

    // ❌ UB: Dangling pointer
    int* dangling;
    {
        int local = 10;
        dangling = &local;
// ...continues on next slide
```

C++ Low-Level Internals and Common Pitfalls

Code Examples

📌 7. Undefined Behavior Examples (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }
    // std::cout << *dangling;  // UB - local is destroyed
}

// ✅ Safe alternatives
void safe_code() {
    // Use smart pointers
    auto p = std::make_unique<int>(42);
    std::cout << *p << "\n";  // Safe

    // Use vector with bounds checking
    std::vector<int> vec = {1, 2, 3, 4, 5};
    try {
        std::cout << vec.at(10);  // Throws exception instead of UB
    } catch (const std::out_of_range& e) {
        std::cout << "Caught: " << e.what() << "\n";
    }
}

int main() {
    safe_code();
}
```

What You Learned

Key Takeaways

- ✓ Object Memory Layout with Vtables
- ✓ Alignment and Padding Optimization
- ✓ Lifetime Management and Dangling References
- ✓ Const Correctness

→ *Swipe to learn* →